

Transaction:

```
CREATE PROCEDURE IF NOT EXISTS AddCourseToSemester(
    IN p_net_id VARCHAR(50),
    IN p_semester VARCHAR(20),
    IN p_year INT,
    IN p_course_code VARCHAR(20)
)
BEGIN
    DECLARE v_net_id VARCHAR(50);
    DECLARE v_user_exists INT;
    DECLARE v_already_planned INT;
    DECLARE v_current_credits INT;
    DECLARE v_course_credits INT;
    DECLARE v_total_credits INT;
    DECLARE v_unsatisfied_groups INT;
    DECLARE v_concurrent_course VARCHAR(20);
    DECLARE v_concurrent_credits INT;
    DECLARE v_concurrent_completed INT DEFAULT 0;
    DECLARE v_concurrent_already_planned INT DEFAULT 0;
    DECLARE v_error_occurred INT DEFAULT 0;

    START TRANSACTION;

    SET v_net_id = p_net_id;

    IF v_error_occurred = 0 THEN
        SELECT COUNT(*) INTO v_already_planned
        FROM Courses_Planned_by_User
        WHERE net_id = v_net_id AND semester_planned = p_semester AND year_planned =
p_year AND course_code = p_course_code;

        IF v_already_planned > 0 THEN
            SET v_error_occurred = 1;
        END IF;
    END IF;

    IF v_error_occurred = 0 THEN
        SELECT credit_hours INTO v_course_credits
```

```

        FROM Course_Information
        WHERE course_code = p_course_code
        LIMIT 1;
    END IF;

    IF v_error_occurred = 0 THEN
        SELECT COALESCE(SUM(ci.credit_hours), 0) INTO v_current_credits
        FROM Courses_Planned_by_User cpb
        LEFT JOIN Course_Information ci ON cpb.course_code = ci.course_code
        WHERE cpb.net_id = v_net_id AND cpb.semester_planned = p_semester AND
cpb.year_planned = p_year;

        SET v_total_credits = v_current_credits + v_course_credits;
    END IF;

    IF v_error_occurred = 0 THEN
        SELECT concurrent_enrollment_course_code INTO v_concurrent_course
        FROM Concurrent_Enrollment
        WHERE course_code = p_course_code
        LIMIT 1;

        IF v_concurrent_course IS NOT NULL THEN
            SELECT COUNT(*) INTO v_concurrent_completed
            FROM Courses_Completed_By_User
            WHERE net_id = v_net_id AND course_code = v_concurrent_course;

            IF v_concurrent_completed = 0 THEN
                SELECT COUNT(*) INTO v_concurrent_already_planned
                FROM Courses_Planned_by_User
                WHERE net_id = v_net_id AND course_code = v_concurrent_course
                AND semester_planned = p_semester AND year_planned = p_year;

                IF v_concurrent_already_planned = 0 THEN
                    SELECT credit_hours INTO v_concurrent_credits
                    FROM Course_Information
                    WHERE course_code = v_concurrent_course
                    LIMIT 1;

                    SET v_total_credits = v_total_credits + v_concurrent_credits;
                END IF;
            END IF;
        END IF;
    END IF;

```

```

END IF;

IF v_error_occurred = 0 AND v_total_credits > 18 THEN
    SET v_error_occurred = 1;
END IF;

IF v_error_occurred = 0 THEN
    SELECT
        COUNT(DISTINCT p.requirement_group_id) INTO v_unsatisfied_groups
    FROM Prerequisite p
    WHERE p.course_code = p_course_code
        AND NOT EXISTS (
            SELECT 1
            FROM Prerequisite p2
            INNER JOIN Courses_Completed_By_User ccu ON p2.prerequisite_course_code =
ccu.course_code AND ccu.net_id = v_net_id
            WHERE p2.course_code = p_course_code
                AND p2.requirement_group_id = p.requirement_group_id
            UNION
            SELECT 1
            FROM Prerequisite p3
            INNER JOIN Courses_Planned_by_User cpb ON p3.prerequisite_course_code =
cpb.course_code
            WHERE p3.course_code = p_course_code
                AND p3.requirement_group_id = p.requirement_group_id
                AND cpb.net_id = v_net_id
                AND (
                    cpb.year_planned < p_year
                    OR (cpb.year_planned = p_year AND (
                        (p_semester = 'Spring' AND cpb.semester_planned = 'Fall')
                        OR (p_semester = 'Fall' AND cpb.semester_planned = 'Spring')
                        OR (p_semester = 'Summer' AND cpb.semester_planned IN ('Fall',
'Spring'))
                    ))
                )
        )
    GROUP BY p.course_code;

    IF v_unsatisfied_groups > 0 THEN
        SET v_error_occurred = 1;
    END IF;
END IF;

```

```

    IF v_error_occurred = 0 THEN
        INSERT INTO Courses_Planned_by_User (net_id, course_code, semester_planned,
year_planned)
            VALUES (v_net_id, p_course_code, p_semester, p_year);

        IF v_concurrent_course IS NOT NULL THEN
            IF v_concurrent_completed = 0 AND v_concurrent_already_planned = 0 THEN
                INSERT INTO Courses_Planned_by_User (net_id, course_code,
semester_planned, year_planned)
                    VALUES (v_net_id, v_concurrent_course, p_semester, p_year);
            END IF;
        END IF;

        COMMIT;
    ELSE
        ROLLBACK;
    END IF;
END

```

Trigger:

```

CREATE TRIGGER remove_planned_after_completion
AFTER INSERT ON Courses_Completed_By_User
FOR EACH ROW
DELETE FROM Courses_Planned_By_User
WHERE net_id = NEW.net_id
AND course_code = NEW.course_code;

```

```

1 CREATE TRIGGER remove_planned_after_completion
2 AFTER INSERT ON Courses_Completed_By_User
3 FOR EACH ROW
4 DELETE FROM Courses_Planned_By_User
5 WHERE net_id = NEW.net_id
6 | AND course_code = NEW.course_code;
7

```

Results

Execution time: 12.1 ms  

 Statement executed successfully

Stored Procedure:

```

CREATE PROCEDURE IF NOT EXISTS GetUserCourses(IN p_net_id VARCHAR(50))
BEGIN
    DECLARE v_user_exists INT DEFAULT 0;

    SELECT COUNT(*) INTO v_user_exists
    FROM User_Table
    WHERE net_id = p_net_id;
    IF v_user_exists > 0 THEN
        SELECT
            cpb.course_code,
            ci.course_name,
            ci.credit_hours,
            cpb.semester_planned,
            cpb.year_planned,
            ROUND(AVG(
                (gpa.percentage_As/100.00 * 4.0) +
                (gpa.percentage_Bs/100.00 * 3.0) +
                (gpa.percentage-Cs/100.00 * 2.0) +
                (gpa.percentage_Ds/100.00 * 1.0)
            ), 2) as avg_course_gpa
        FROM Courses_Planned_by_User cpb
        LEFT JOIN Course_Information ci ON cpb.course_code = ci.course_code
        LEFT JOIN course_gpa_by_instructor gpa ON cpb.course_code = gpa.Course_Code AND
gpa.Total_Students > 0
        WHERE cpb.net_id = p_net_id
        GROUP BY cpb.course_code, ci.course_name, ci.credit_hours,
            cpb.semester_planned, cpb.year_planned
    
```

```

ORDER BY cpb.year_planned,
        CASE cpb.semester_planned
            WHEN 'Fall' THEN 1
            WHEN 'Spring' THEN 2
            WHEN 'Summer' THEN 3
        END,
        cpb.course_code;

SELECT
    ccu.course_code,
    ci.course_name,
    ci.credit_hours,
    ccu.semester_taken as semester_completed,
    ccu.year_taken as year_completed,
    COUNT(DISTINCT p.course_code) as prereq_count,
    COUNT(DISTINCT ce.course_code) as concurrent_count
FROM Courses_Completed_By_User ccu
LEFT JOIN Course_Information ci ON ccu.course_code = ci.course_code
LEFT JOIN Prerequisite p ON ccu.course_code = p.prerequisite_course_code
LEFT JOIN Concurrent_Enrollment ce ON ccu.course_code =
ce.concurrent_enrollment_course_code
WHERE ccu.net_id = p_net_id
GROUP BY ccu.course_code, ci.course_name, ci.credit_hours,
        ccu.semester_taken, ccu.year_taken
ORDER BY ccu.year_taken DESC,
        CASE ccu.semester_taken
            WHEN 'Fall' THEN 1
            WHEN 'Spring' THEN 2
            WHEN 'Summer' THEN 3
        END,
        ccu.course_code;
END IF;
END

```